Chatbot KnowledgeBase

Provisioning Approaches Compared

Approach	Details
EDAV BaaS (Bot as a Service) / Microbot	• **What the provider gives us:** A ready-made bot and its built-in search features. Exactly what it can do depends on EDAV. • **What we set up:** The approved content, the bot's instructions, answer checks, and (where possible) embedding it in our app with login. • **Knowledge formats:** Word/PDF, per EDAV's guidance. Turn approved Excel data into readable documents. • **Short-term fit:** Least work to build, if we get access in time. Past trouble embedding it (iframe issues) is the main schedule risk. • **How embedding works:** The bot loads inside our page in an *iframe* (a framed window-within-a-page), and the two communicate through *postMessage*, the browser's secure way to pass data between a page and an embedded frame.
EDAV MaaS (Model as a Service)	• **What the provider gives us:** Access to a hosted model. Any extra tools depend on the service. • **What we set up:** The chat screen, backend calls, picking and fetching sources, citations, access checks, error handling, and testing. • **Knowledge formats:** Markdown/text as context, plus Word/PDF we parse and controlled SQL results. • **Short-term fit:** More work, but more control. A good fit if we have a working endpoint but can't use Microbot.
Direct OpenAI API with hosted file search	• **What the provider gives us:** Model access plus a built-in file search. • **What we set up:** The chat and backend, file uploads, source permissions, showing citations, testing, and any SQL tools. • **Knowledge formats:** The built-in file search handles DOCX, PDF, Markdown, and more. • **Short-term fit:** Moderate work if we're already approved to use it. Getting a new provider approved could cost more time than it saves.
Model API with fully custom retrieval	• **What the provider gives us:** Just the model itself. • **What we set up:** Everything — reading documents, indexing, search, ranking, SQL tools, source filtering, citations, chat, and upkeep. • **Knowledge formats:** Documents plus structured database data. • **Short-term fit:** The most work. Skip it for the short term unless we already have the pieces built.

> In the table below, **"Ideal"** means the recommended long-term design. **"Short-term"** means the faster, lighter version (such as for the buildathon), using versioned reference snapshots rather than live database integration.

Knowledge Areas (by subject domain)

Knowledge area	Brief description	Ideal (long-term) format, storage & retrieval	Short-term format & retrieval	Questions it could answer
1. Competency requirements	Governing lab requirements	Approved **Word/PDF** in controlled storage; **document RAG** filtered by source/version	Approved **Word/PDF** loaded into EDAV's bot	① What requirements apply? ② Which elements are required? ③ Where is this requirement documented?
2. Assessment form instructions	Field meanings and what to document	Approved **Word/PDF**, retrieved by form version/section; form fields stay in **Azure SQL**	A short **Word/PDF form guide** organized by field name	① What does this field mean? ② Which fields are required? ③ What belongs in evidence/comments?
3. Approved test systems	Catalog entries and groupings	**Azure SQL** data, read via exact read-only lookups	A dated **Word/PDF catalog snapshot** keeping each entry's name, grouping, and status together	① Which systems are active in this snapshot? ② Which grouping contains this system? ③ What is its approved name?
4. Assessment workflow	Work states and sequence	Maintained **Markdown** as context or RAG; **Python** enforces rules, **Azure SQL** stores state	Workflow guide exported to **Word/PDF** for EDAV	① What follows submission? ② What does Returned mean? ③ What precedes finalization?
5. Roles and responsibilities	Duties and authority limits	Maintained **Markdown** role guide as context; app authorization stays separate	A **Word/PDF** role guide listing duties and limits	① What does an Assessor do? ② How do Team Lead and Supervisor differ? ③ Can an administrator sign?

Knowledge area	Brief description	Ideal (long-term) format, storage & retrieval	Short-term format & retrieval	Questions it could answer
6. Evidence and follow-up requirements	Evidence, gaps, and remedial actions	Approved **Word/PDF** via **document RAG**; individual records kept out of general retrieval	Approved **Word/PDF**, organized by assessment element	① What evidence is required? ② What documents an unsuccessful element? ③ What follow-up is needed?
7. Due dates and renewal	Intervals and how dates are figured	**Azure SQL** settings; **Python** does the exact math; Markdown explains it	A dated **Word/PDF** interval reference; the app still calculates official dates	① What interval applies to this grouping? ② Which date starts renewal? ③ What does overdue mean?
8. Signatures and finalized records	Signing order, locking, versioning	Approved **Word/PDF** via RAG; **Azure SQL/Blob** records stay out of the corpus	A short approved **Word/PDF** procedure	① Who signs first? ② Why is a finalized record locked? ③ What happens after an edit?
9. Application help	Navigation and reporting	Versioned **Markdown** help; RAG or context by size	Help exported to **Word/PDF**, matching the demo app version	① Where are my assignments? ② Where are items awaiting signature? ③ How do I open reports?
10. Terminology	Controlled definitions	**Markdown** glossary; exact lookup or small context	A **Word/PDF** glossary, one labeled definition per entry	① What means Testing Personnel? ② How do systems and groupings differ? ③ What is an amendment?
11. Support and escalation	Approved help routes	Maintained **Markdown** directory; routes chosen deterministically	A **Word/PDF** support guide mapping issues to contacts	① Who clarifies requirements? ② Where do I report access issues? ③ Who reviews conflicting guidance?

Knowledge area	Brief description	Ideal (long-term) format, storage & retrieval	Short-term format & retrieval	Questions it could answer
12. Approved-source registry	Source versions, owners, allowed use	**Structured registry** (ideally Azure SQL); filtering enforced before retrieval	A **Markdown** manifest for maintainers, source details inside each file, and a manually controlled upload set	① Which source supports this answer? ② What version is it? ③ Who owns it?